Thought Anchors — Implementation & Execution Specification

Companion build spec to the research protocol
(thought_ anchors_ experiments_ v3)

1 Purpose and how to use this document

This is a *build specification*, not source code. It states what code *should* exist—module responsibilities, data schemas, interface signatures, algorithms in sketch form, smoke-test assertions, the parallel/sequential phase structure, the cluster run model, the per-result decision logic, and the human breakpoints—at a level explicit enough to implement directly, without committing the implementations yet. It covers everything in the protocol *through the trials/tiers*; the causal interventions (Q1–Q4) are out of scope here and built only after a detector is validated.

The two-layer build philosophy (read first). The document is written all at once so that early plumbing anticipates later plumbing; the *code* is built in two layers:

- **Foundation (§4): build fully, up front, against forward-looking interfaces.** The shared spine—schema, storage, checkpointing, splits, raw-rollout retention, config assertion, smoke harness. This is where "no leak / no mid-run crash / launch-and-forget" is won or lost, so it is designed with the *whole* pipeline in view, including signals not computed until later tiers.
- **Tier modules (§5, §7): specify now, generate and run on demand at the gates.** The tiered/fork design exists precisely so machinery is added only as evidence demands. Build a whole tier, run it, decide at the breakpoint; do not work ahead.

"Wait for results" means wait at the **breakpoints** (§9), not after every module.

2 Repository layout

```
ta/
  config/        run configs (yaml): model, hyperparams, phase, tier, seeds
  core/
    modelio.py   M0  load + config assertion
    traces.py    M1  base CoT trace generation
    spans.py     M2  clause-respecting span segmentation
    signals/     M3  one file per signal family (force, katz, mlp_g, grad_d, online)
    labels.py    M4  resample-and-filter gold labels (+ constructed-swap check)
    combiners.py M5  ridge / PCR / MLP
    metrics.py   M6  spearman, bootstrap CI, receiver-head baseline, decision logic
  infra/
    schema.py        dataclasses + (de)serialization for all artifacts
    store.py         atomic keyed read/write, content/config hashing
    queue.py         file-based work queue, claim/complete sentinels
    seeds.py         deterministic seed derivation
```

```
    splits.py        grouped-by-problem split assignment + leak assertions
    manifest.py      run manifest (git sha, config hash, model facts)
  smoke/         one test module per core module + integration tests
  run/
    launch_phase.sh  tmux launcher: one worker pane per GPU + monitor
    worker.py        idempotent unit processor (pulls from queue)
    summarize.py     end-of-phase decision report + DONE sentinel
  data/          raw datasets + adapters (babi, bbh_shuffled, clutrr, scone, gsm8k)
  artifacts/     all outputs, sharded by phase/dataset/problem_id (gitignored)
```

3 Environment and dependencies

- Python 3.11; `torch` (CUDA build matching the cluster driver), `transformers`, `accelerate`, `safetensors`, `numpy`, `scipy` (Spearman, Fisher-z), `scikit-learn` (ridge/PCR/MLP, grouped CV), `pyarrow` (parquet), `pyyaml`.
- Two model handles (§5.1): an *eager*-attention handle for the attribution pass (needed to materialise attention probabilities) and an SDPA/flash handle for the resampling rollouts.
- Pin all versions in a lockfile; record the lockfile hash in the run manifest. Transformers internals (attention return signature, RoPE, `repeat_kv`) are version-sensitive; assert against them in M0 rather than trusting the version string.

4 Foundation: cross-cutting design (build this first, completely)

Everything here is shared infrastructure with high change-cost. Implement and smoke-test it before any signal or label code, against interfaces that already include the not-yet-built signals.

4.1 Artifact schema (`infra/schema.py`)

Forward-looking: the `Signals` record reserves fields for *every* signal, even those a given run does not populate, so later tiers never reshape storage.

```
Problem:   problem_id, dataset, phase, prompt, gold_answer,
           vocabulary: {type -> [valid values]},   # typed closed vocab (structured tasks)
           supporting_facts: [token-spans]|None,     # bAbI free ground truth
           meta
Trace:     trace_id, problem_id, token_ids, text, gen_seed, gen_config_hash
Span:      span_id, trace_id, clause_id, tok_start, tok_end, text,
           is_window: bool, task_tokens: [idx], frame_tokens: [idx]
Signals:   span_id ->
             E, P, Pchained,          # routing (one-hop, Katz)
             G,                        # computation (MLP write magnitude)
             Pprime, PprimeChained,    # crude computation-weighted outflow
             D, Dchained, D_ig,        # gradient-scaled outflow (+ chained, + IG)
             c, position, length,      # online, confounds
             meta: {band, K, lambda, g, sigma_pop}
Label:     span_id ->
             A,                        # gold importance (Eq. label)
             real_dist: {answer->p}, diff_dist: {answer->p},
             raw_rollouts: [{cond: real|diff, answer, parse_ok, filter_sim}],  # RETAINED
             R_real, R_diff_net, R_gross, parse_fail_rate, metric: tv|kl|flip
ConstructReport: span_id -> {diff_shift, similar_shift, has_task_tokens}
```

```
SplitAssignment: problem_id -> {fold, role: train|test}
RunManifest: git_sha, config_hash, lockfile_hash, model_facts(M0), seeds, phase, tier, started_at
```

Critical no-leak decision: `Label.raw_rollouts` is retained in full. The expensive artifact is the rollouts, not A; storing per-rollout answers means A (and the metric choice, and the filter threshold) can be recomputed offline forever *without re-rolling*. Never store only the scalar A.

4.2 Storage (`infra/store.py`)

- Sharded by `phase/dataset/problem_id`; tabular as parquet/jsonl, tensors as safetensors.
- **Atomic writes**: write to `*.tmp` then `rename` (POSIX atomic); a half-written file can never be read as complete.
- **Content/config keying**: each artifact's path embeds a hash of its inputs + config. A rerun with identical inputs/config is a no-op (idempotent); a changed hyperparameter writes a new key rather than silently overwriting.
- `read(key)` raises on absent (callers distinguish "not computed" from "empty").

4.3 Checkpointing, idempotency, resumability (`infra/queue.py`)

- Unit of work = the smallest re-runnable chunk: one *problem* for trace gen / signal extraction; one *span* (or small span-batch) for rollouts (rollouts dominate cost, so checkpoint finely there).
- File-based queue: `claim(unit)` writes a `.claimed` lease (with PID + timestamp); `complete(unit)` writes a `.done` sentinel. On restart a worker skips `.done` units and reclaims stale `.claimed` leases (PID dead or lease older than T).
- Consequence: kill the job at any point and relaunch; it resumes from the last `.done` with no recompute and no corruption. This is the launch-and-forget guarantee, and it is smoke-tested (§6, resume-after-kill).

4.4 Determinism and seeding (`infra/seeds.py`)

```
seed(purpose, problem_id) -> int   # = hash(global_seed, purpose, problem_id) mod 2^31
```

All generation/sampling draws its seed this way, so any unit reproduces identically on rerun. Record the global seed in the manifest. Note throughput-vs-bitwise-determinism: set deterministic kernels only for the smoke tests that assert bitwise equality; production rollouts may relax this for speed but remain *seeded* (distributions reproduce, not necessarily bitwise).

4.5 Grouped splits and leak prevention (`infra/splits.py`)

- Splits are assigned at *problem* granularity; a problem's every span shares its problem's fold/role. Within-trace spans are correlated, so splitting them across train/test leaks.
- `assert_no_leak(split)`: train and test `problem_id` sets are disjoint; fails the run loudly if violated. Called before any combiner fit and inside the combiner smoke test.
- Effective sample size for all CIs is the *number of traces/problems*, not spans.

4.6 Config assertion as a hard gate (`infra/manifest.py` + M0)

The architecture corrections in the protocol all hinge on the exact model config; assert them, do not trust them. See §5.1.

4.7 Smoke-test harness (`smoke/`)

A single runner discovers and runs all smoke tests, prints a pass/fail table, and **blocks phase launch** unless the phase's required tests pass. Every module ships its smoke tests with it. The harness distinguishes *correctness* tests (must pass to proceed) from *calibration* reports (inform breakpoints).

5 Module specifications

Each module lists: responsibility, interface, key correctness points (from the protocol), and smoke tests. Interfaces are signatures, not implementations.

5.1 M0 — Model loading and config assertion (`core/modelio.py`)

Responsibility. Load the two model handles; extract and assert architecture facts; expose the static weights the signal code needs (W_O per-head blocks, RMSNorm gains).

```
load_model(path, attn_impl: "eager"|"sdpa") -> (model, tokenizer)
assert_config(model.config) -> ModelFacts
  asserts: num_hidden_layers == 28
           hidden_size == 1536 ; head_dim == 128
           num_attention_heads (H) and num_key_value_heads (Hkv) present
           group_size = H // Hkv  (record; GQA expected, Hkv < H)
           norm type is RMSNorm ; record rms_norm_eps
           rope present ; record theta
           eager handle actually returns attention probs on a 4-token probe
ModelFacts: {L,H,Hkv,group_size,d,d_head,eps,...}  # written to manifest
wo_head_blocks(model) -> Tensor[L, H, d_head, d]   # o_proj reshaped per query head
rms_gains(model) -> {input_ln[L], post_attn_ln[L]}  # gamma vectors
```

Correctness. Fail loudly on any mismatch — each is a silent-corruption risk (GQA mis-grouping, wrong norm linearisation, attention probs silently empty under SDPA). This runs as step 1 of every phase.

Smoke tests. (i) assert passes on the real model; (ii) assert *raises* on a synthetic config with wrong head counts / norm type; (iii) eager probe returns attention of shape $[H, T, T]$; (iv) `wo_head_blocks` recomposed equals the original `o_proj` weight.

5.2 M1 — Base trace generation (`core/traces.py`)

Responsibility. Generate the CoT traces that are the substrate for spans, signals, and labels.

```
generate_trace(problem, model_fast, gen_config, seed) -> Trace
  # few-shot prompt forcing a parseable final answer ("Answer: X")
  # temperature, max_new_tokens, stop conditions in gen_config
```

Correctness. Deterministic per `seed(problem_id,"trace")`; store full token_ids (spans index into these). Record the answer-format scaffold per dataset (M4 parses it).

Smoke tests. 5 bAbI problems generate, parse to a valid answer, and round-trip through store/load unchanged; rerun with same seed is bitwise identical.

5.3 M2 — Clause-respecting span segmentation (`core/spans.py`)

Responsibility. Two-stage boundary-first segmentation (protocol §Granularity): parse clauses, then window within a clause.

```
segment(trace, tokenizer, t, clause_rules) -> [Span]
  # stage 1: split decoded text at sentence terminators, newlines,
  #          comma clause-boundaries, connectives {so,therefore,thus,then,next,because}
  # stage 2: map each clause's char span -> token span via offset_mapping
  #          (tokenizer(..., return_offsets_mapping=True));
  #          clause <= t tokens -> 1 span; longer -> consecutive t-token windows
  #          (final remainder merged into previous if < t/2). Never cross a clause boundary.
tag_vocab(span, problem.vocabulary) -> (task_tokens, frame_tokens)  # for construct check
```

Correctness. The text↔token alignment is the subtle part — use offset mappings, never re-tokenise substrings. Spans partition the trace (every non-special token in exactly one span).

Smoke tests. (i) spans never cross a clause boundary; (ii) decoded span tokens equal the clause substring they came from (alignment); (iii) union of spans = full trace, disjoint; (iv) windows respect t and the merge rule.

5.4 M3 — Signal extraction, the attribution pass (`core/signals/`)

Responsibility. One eager forward pass over the base trace, plus per-layer local backward passes, producing all forward-time signals at span granularity.

Delivered force and edges (`force.py`).

```
delivered_force_norms(trace, model_eager, facts) -> Ehat[T,T], (per-target topK survivors)
  # per query head h: F^{l,h}_{s->t} = WO_block[h] @ (alpha^{l,h}_{t,s} * v^{kv(h)}_s)
  #   GQA: kv(h) = h // group_size ; value from shared kv head
  #   pre-norm: F added to residual with NO post-attn norm -> exactly additive, no bias
  # pruning: Ehat_{s->t} = sum_{l,h} alpha * ||WO_block[h] @ v_s||  (cheap upper bound)
  #   keep top-K sources per target t -> N(t); compute exact ||F|| only on survivors
edges(...) -> E[span,span]   # E_{s->t} = sum_l ||F^l_{s->t}||, aggregated to spans
```

Correctness: GQA mapping; W_O per-head block; do *not* apply any post-attention norm (pre-norm). Pruning is top-K incoming per target by the routing bound $\widehat{E}$; with bounded g this also covers the one-hop computation signals (use bounded g, below).

Katz routing propagation (`katz.py`).

```
katz(E, lambda) -> P[span]   # right-to-left DP on the causal DAG:
                             # P_s = sum_{t>s} E_{s->t} (1 + lambda * P_t)
```

DAG (strict $t > s$) guarantees termination; sweep λ later (Tier 3). One-hop E-outflow is λ=0.

Computation magnitude (`mlp_g.py`).

```
mlp_write_G(trace, model_eager, band B, sigma_pop) -> G[span]
  # per layer l in B: g_l = || MLP_l( RMSNorm_postattn(h'_l) ) ||   (down_proj output, SwiGLU)
  # G_t = sum_{l in B} g_l / sigma_l ;  G_span = sum_{t in span} G_t
  # sigma_l = cross-position std at layer l, computed over sigma_pop (DECIDE: pooled vs per-trace),

  #           treated as a frozen constant (detached)
```

Correctness: in pre-norm ‖MLP output‖ *is* the net residual write — do not compute a separate "net write". Band B localised in Phase 0.

Gradient-scaled attribution (`grad_d.py`).

```
grad_D(trace, model_eager, band B, sigma) -> D[span], (optional) D_ig[span]
  # per layer l in B (LOCAL Jacobian, not full-network gradient):
  #   treat pre-MLP residual h'_l as a detached leaf requiring grad
  #   forward through layer-l MLP sub-block (incl. its input RMSNorm)
  #   loss = sum_t || MLP_l ||  (sum over positions; position-wise => no cross-contamination)
  #   backward once -> grad_{h'_l}[t] = d||MLP_l||/d h'_{l,t}   for all t
  #   D^l_{s->t} = (grad_{h'_l}[t] / sigma_l) . F^l_{s->t}
  # D_{s->t} = sum_{l in B} D^l ;  aggregate to spans like E
  # D_ig: replace point gradient by integral over alpha in [0,1] of grad at h'_t - (1-alpha) F_{s
    ->t}
  #       (baseline h'_t - F_{s->t}, shared with Taylor and exact ablation)
```

Correctness: differentiate w.r.t. the *residual* h' (RMSNorm in the autodiff path) so it composes with F; one backward *per layer* in B, position-vectorised; σ detached. Cost $\approx |B|$ local backwards.

Computation-weighted outflow and chained variants (`outflow.py`).

```
pprime(E, G, g) -> Pprime[span]            # P'_i = sum_{j>i} E_{i->j} g(G_j)
chained(E_or_D, G, g, lambda) -> [span]    # Tier-3: Katz-style recursion carrying g(G)
online_c(N(t)) -> c[span]                  # streaming in-degree (# targets that kept i)
position, length -> from spans
```

Correctness: restrict g to *bounded* forms (log / centred / threshold) so routing-pruning suffices and chained variants do not blow up; chaining is Tier-3 (improvement) or a Tier-2 rescue, not Tier-1.

M3 smoke tests (the correctness core — gate the whole project).

- **Force additivity (the key test).** Reconstruct each target's pre-MLP residual from the per-source forces: $h_{\text{block-in}} + \sum_s F_{s\to t} \overset{?}{\approx} h'_t$ (actual residual), within fp tolerance. Passing validates GQA grouping, W_O blocks, α capture, and pre-norm additivity *simultaneously*.
- **GQA mapping.** Per-head F on a tiny input matches a hand-computed reference.
- **Gradient finite-difference.** $\nabla G \cdot \delta \approx G(h'+\delta) - G(h')$ for small random δ (validates the local Jacobian).
- **D vs exact ablation** on a span subsample: $D_{s\to t}$ correlates with $G_t(h'_t) - G_t(h'_t - F_{s\to t})$ — so a future D-null is read as "no signal" not "bad approximation".
- **Katz vs brute force** on a tiny hand-built DAG.
- **Pruning soundness**: $\widehat{E} \geq \|F\|$ elementwise (valid upper bound).

5.5 M4 — Resampling labels (`core/labels.py`)

Responsibility. Produce the gold importance A by the resample-and-filter method of the protocol; provide the constructed-swap construct-validity check; retain raw rollouts.

```
resample_filter(trace, span, model_fast, R, filter, metric, seed) -> Label
  # real:  K_real continuations of the unperturbed prefix-through-i
  # diff:  resample replacement spans for i FROM THE MODEL (prefix up to start of i),
  #        filter for semantically-DIFFERENT (filter.sim(orig,repl) < tau), keep until R_diff net;
  #        continue CoT to answer from each
  # parse each continuation -> discrete answer (per-dataset parser); drop unparseable (record rate
    )
  # A = metric( real_dist, diff_dist )   # TV default; match theirs (tv|kl|flip)
  # store ALL raw_rollouts (cond, answer, parse_ok, filter_sim)
```

```
filter: SemanticFilter   # FROM BOGDAN ET AL. CODE if reused; else reimplement (embed/NLI + tau)
parse_answer(text, dataset) -> answer|None
constructed_swap_check(trace, span, problem.vocabulary) -> ConstructReport
  # different = swap a task-content token to same-type sibling; similar = edit frame tokens only
  # report diff_shift (should be high on anchors) and similar_shift (should be ~0)
```

Correctness / decisions.

- **Reuse first (Tier-0).** If Bogdan et al.'s released labels cover the model, ingest them and skip generation entirely — this fixes the filter, the metric, and the hyperparameters, and sidesteps the granularity question. Only reimplement if reuse fails.
- **Filter / metric / hyperparameters** (`tau`, oversampling factor to net R different, K_{real}, temperature, max-continuation) are read from their code; do not invent.
- **Granularity transfer.** Their filter is sentence-tuned; in Phase 0 confirm it calibrates on clause-windows (semantically-different resamples of a short span are obtainable). If not, fall back to sentence granularity (which aligns with reusing their dataset).
- **Rollouts on the SDPA/flash handle** (fast); the eager handle is only for M3.

M4 smoke tests.

- **Real-vs-real null (the key test).** With no perturbation, $A \approx 0$ — establishes the label noise floor so any $A > 0$ is signal, not pipeline artifact.
- **Constructed-swap behaviour** on bAbI: `diff_shift` large on annotated supporting facts, `similar_shift`≈ 0; a real-vs-real null per arm.
- **Parser accuracy** $\geq$ threshold on a hand-labelled sample; `parse_fail_rate` recorded; require a minimum count of valid rollouts per condition or discard the span.
- **Raw-rollout recompute**: recomputing A from stored `raw_rollouts` reproduces the stored A (so labels never need regeneration).
- **Filter sanity**: known paraphrase pairs classified similar, known content-swaps different.

5.6 M5 — Combiners (`core/combiners.py`) [Tier 2+]

Responsibility. Combine signals into a predictor; the interpretable model's coefficients are the deliverable, the MLP is an accuracy ceiling.

```
fit_ridge(signals, labels, split) -> {coef, rho_test, ci}        # alpha by grouped CV
fit_pcr(signals, labels, split) -> {...}                          # SVD principal-component
    regression
fit_mlp(signals, labels, split) -> {rho_test, ci}                 # 2-layer, width 32-64, early
    stop
```

Correctness. `assert_no_leak` on the split before every fit; grouped K-fold by problem; report on held-out problems only; keep the outflow signal only if it adds over P+G.

Smoke tests. grouped split shares no problem across train/test; ridge recovers known coefficients on synthetic data; reruns reproduce.

5.7 M6 — Metrics, baseline, decision logic (`core/metrics.py`)

Responsibility. Score signals against A, reproduce the receiver-head baseline, apply the two-pronged decision.

```
spearman_vs_A(signal, labels, by_trace) -> (rho, ci)   # bootstrap over TRACES; Fisher-z
receiver_head_baseline(trace, model_eager) -> scores    # reuse Bogdan et al. code if available
tier1_decision(rhos) -> {prong_i: any rho>0.22?,
                         prong_ii: max(rho_G, rho_Pprime) > rho_P (or adds over P)?,
                         fork: improvement|fix}
```

Correctness. Effective N = number of traces; pre-declare the confirmatory comparisons and the multiple-comparison correction (FDR over the marginals) separately from exploratory sweeps.

Smoke tests. Spearman matches `scipy` on known data; bootstrap CI has nominal coverage on synthetic data with known ρ; baseline reproduces a known value on a fixture trace.

6 Smoke-test suite (consolidated)

Correctness tests block launch; calibration reports inform breakpoints. Beyond the per-module tests above, the integration tests:

- **End-to-end dry run** on 5 problems: traces → spans → signals → small-R labels → one ridge fit, all artifacts written and reloadable, no exception.
- **Resume-after-kill**: launch the dry run, `SIGKILL` a worker mid-rollout, relaunch; assert (a) no `.done` unit recomputed, (b) no corrupt/half-written artifact, (c) final result identical to an uninterrupted run.
- **Leak audit**: across the whole pipeline, no `problem_id` appears in two folds; no span's label rollout reuses its own continuation as a "different" sample.
- **Determinism**: two seeded dry runs produce identical artifact hashes (under deterministic kernels).
- **Resource guards**: pre-flight estimates artifact bytes and peak GPU memory from (#problems, trace length, R, T); asserts free disk and fits batch to memory; refuses launch otherwise.
- **Schema round-trip**: every artifact type serialises and deserialises to an equal object, including the reserved-but-empty signal fields.

Rule: a phase launches only when its required correctness tests are green. This is what makes "launch and walk away" safe.

7 Phased execution and the cluster run model

Tiers (*what* to run) and curriculum phases (*which* data) are orthogonal. Tier 0/1 run on the Phase-0/1 data first; later tiers extend to later phases. Within any cluster run:

Parallel vs sequential.

- **Embarrassingly parallel** (one queue unit each, fan out across all 8 GPUs): trace generation (per problem), signal extraction (per trace), and—dominant—resampling rollouts (per span). These never need to talk to each other.

- **Sequential barriers**: segmentation after traces; signals and labels may run concurrently (both depend only on traces); analysis after both; band/t/K localisation needs a *first small label batch*, so a short sequential pre-batch precedes the big parallel labeling.

The tmux launch-and-forget model (`run/`).

```
launch_phase.sh <phase> <tier>:
  1. run required smoke tests; abort if any correctness test fails
  2. build the work queue for the phase (list of unit ids; skip .done)
  3. start a tmux session:
       - one pane per GPU: worker.py --gpu k   (CUDA_VISIBLE_DEVICES=k; pull/claim/process/
     complete)
       - one monitor pane: progress, ETA, failure tail, queue depth
  4. on empty queue: summarize.py writes the decision report + a DONE sentinel
worker.py: idempotent; on crash, lease expires and another worker reclaims the unit; checkpoints
     per unit
```

You launch, detach tmux, and leave. Workers are idempotent and resumable, artifacts are atomic and keyed, so a node failure or preemption costs at most the in-flight units, never a re-run. The DONE sentinel + report tell you when to come back.

Run-phase mapping.

1. **Build phase (local, sequential).** Implement the Foundation; pass all correctness smoke tests; end-to-end dry run. ⇒ **Breakpoint 1**.

2. **Tier-0 gates (cheap, mostly local).** Reuse check; free bAbI construct check; small-R pilot *that doubles as* t/K/band-B localisation and the granularity-transfer check. ⇒ **Breakpoint 2**.

3. **Tier-1 run (cluster, parallel).** Full R labeling + P, G, P' marginals + receiver-head baseline on Phase-0/1. ⇒ **Breakpoint 3 (the fork)**.

4. **Tier-2/3 (cluster, on demand).** Improvement (or fix) features per the fork; combiners; chaining/reach; extend phases. ⇒ **Breakpoint 4**.

8 Decision logic — what to do per result

Mirrors the protocol's tiers; computed by `tier1_decision` and the gate scripts.

- **Tier-0 reuse.** Covered ⇒ ingest their labels, skip generation, proceed to Tier-1 marginals. Not covered ⇒ reimplement filter/metric from their code.

- **Tier-0 construct check.** Signals rank supporting facts above distractors ⇒ encouraging, proceed. They do not ⇒ strong warning; inspect signal code before spending the labeling budget (leading indicator, not a hard stop).

- **Tier-0 pilot.** Even noisy ρ shows nothing on any signal ⇒ stop and re-examine before full R. Granularity-transfer fails ⇒ fall back to sentence granularity (and prefer reuse).

- **Tier-1 two-pronged.** (i) some marginal > 0.22? (ii) does G or P' beat / add over P?
  - both yes ⇒ *improvement*: add D (Trial 3) and the linear combiner; keep outflow only if it adds over P+G.

  - routing clears but computation does not (or position-dominated) ⇒ *fix*: rule out (a) position confound (residualise), (b) bad approximation (D^{IG}), (c) wrong reach (pull the chained variant forward as a rescue) before declaring the axis inert.
  - nothing clears 0.22 ⇒ fix branch, then if still null, a clean negative result.

- **Tier-3.** Chained underperforms or position-dominated ⇒ remediation ladder (centre $g \to$ per-hop cap $\to$ sub-critical $\lambda \to$ row-normalise; residualise position). MLP ceiling reported beside, never instead of, the interpretable model.

9 Human breakpoints

Stop, inspect code and results, and decide — do not let the pipeline auto-continue past these.

1. **After the Foundation (before any expensive labeling).** Review the correctness core: force-additivity, gradient finite-difference, D-vs-ablation, real-vs-real null. *Question:* is the machinery provably correct? Nothing costly runs until yes.

2. **After Tier-0 gates.** Review reuse availability, the construct-check ranking, the pilot ρ, and the chosen $t/K/B$ and granularity-transfer verdict. *Question:* are inputs and hyperparameters sane, and is the full labeling budget worth committing?

3. **After Tier-1 marginals (the fork).** Review the two-pronged decision. *Question:* did the fulcrum clear 0.22, and did the *computation axis* pay off (vs. a routing-only win)? Choose the improvement or fix branch here.

4. **After the first combiner / chained results.** *Question:* is the gain real, or position/overfit? Confirm on held-out problems and against the position-only null before scaling phases.

10 Build-and-run checklist

1. Implement `infra/` (schema, store, queue, seeds, splits, manifest) + smoke harness.

2. Implement M0; pass config-assertion tests.

3. Implement M1, M2, M3; pass the correctness core (force additivity, gradient FD, Katz, pruning).

4. Implement M4 (with constructed-swap check) + M6 baseline/metrics; pass real-vs-real null and parser tests. *End-to-end dry run.* $\to$ **Breakpoint 1**.

5. Tier-0: reuse check $\to$ construct check $\to$ small-R pilot/localisation. $\to$ **Breakpoint 2**.

6. Launch Tier-1 on the cluster (tmux). $\to$ **Breakpoint 3**.

7. Implement M5 and the fork-selected features; launch Tier-2/3 on demand. $\to$ **Breakpoint 4**.

Causal interventions (Q1–Q4) are deliberately out of scope until a detector is validated.